使用说明 · 校园 AI 助手（campus-ai）

项	内容
作品名称	校园 AI 助手（campus-ai）
构建号	`2026-11-01-a`
运行环境	Windows + Python 3.9 及以上（无需 Node、无需数据库服务）
交付形态	**解压即用** —— 大模型 / 地图 / 向量库的 API Key 均已配置在 `backend/.env`

> **【注意】本包内含真实 API Key**（`backend/.env`）。请勿上传至公开仓库或公开分享。
> 换学校部署时，照 `backend/.env.example` 换成自己的 Key 即可。

一、怎么打开网页（先看这里）

先登录 · 默认管理员账号

第一次打开是登录页。数据库在**首次启动时自动创建**，并写入一个默认管理员账号：

项	值
用户名	`admin`
密码	`zrc666`
角色	管理员（`is_admin = 1`）

用它可以立刻看到两个管理页面：

- **运营数据大屏** —— `/app/admin.html`（用户数、课表、情绪、反馈四组真实统计）
- **收到的反馈** —— `/app/admin-feedback.html`（列表 + 回复）

管理员的顶栏只有「数据」和「反馈」两项 —— 主页 / 学习规划 / 情绪陪伴 / 地图导航 是给学生用的，管理员不需要。想看看学生那边长什么样，进「我的」页点「进入用户视图」。

> 学生账号请点登录页的「注册」自己建。
> **全站统计一律排除管理员**（管理员不算「用户」，不然大屏上的用户数永远自带 1）。
> 改管理员密码：登录后进「我的 → 修改密码」。
> 管理员账号只在**库为空 / 没有同名账号**时创建一次，重启不会重置密码。

界面偏好：顶栏靠右有主题开关，可选 **跟随系统 / 白天 / 护眼 / 黑夜。** 选择后立即生效并持久化；选黑夜时地图底图也会跟着切成深色。首次进入时，页面在骨架屏 （灰色微光占位）短暂出现后才显示真实数据 —— 那是加载中的样子，不是卡住。

方式 A • 公网直接访问（无需在本地安装任何东西）

```
https://campus-ai-assistant-61569.app.workbuddy.host/
```

已在云端常驻运行，打开即为登录页（根路径会自动跳到登录页）。

【注意】云端静态页面在 `/app/` 前缀下，直接手敲地址时请带上，例如：

```
https://campus-ai-assistant-61569.app.workbuddy.host/app/login.html
https://campus-ai-assistant-61569.app.workbuddy.host/app/planner.html
```

方式 B • 本地一键启动（约 30 秒）

1. 把压缩包解压到任意目录（路径**不含空格**更省心）
2. 双击根目录的 `start-windows.bat`
3. 脚本自动完成：检测 Python → 补装后端依赖（首次约 1–2 分钟）→ 启动后端 → 启动前端 → **自动打开浏览器**

浏览器会打开：`http://127.0.0.1:5500/login.html`

> **双击后黑窗口一闪就没？** 说明 Python 没装或不在 PATH 里。
> 双击 `check-windows.bat` 看具体缺什么 —— 它只输出信息，不改动任何东西。

二、全部访问地址一览

一键启动后，脚本会在窗口里打印下面这组地址：

用途	地址	说明
电脑 · 前端端口（脚本自动打开的）	`http://127.0.0.1:5500/login.html`	前后端分开起，改前端代码刷新即见
电脑 · 单端口（推荐，更省事）	`http://127.0.0.1:8000/app/login.html`	**只跑后端就能用**，静态页由后端托管
手机 · 同一 Wi-Fi	`http://<电脑IP>:8000/app/login.html`	电脑 IP 脚本会打印；手机不再分两套话术，用浏览器打开即可
接口调试文档	`http://127.0.0.1:8000/docs`	FastAPI 自动生成的 Swagger

> **端口 8000 / 5500 被占用？** 脚本会自动在 `8000-8020` / `5500-5520` 里挑空闲端口，并把最终端口打印在窗口里 —— 以它打印的为准。
>
> **【注意】走单端口方式时一定要带 `/app/` 前缀**（如 `/app/login.html`）。
> 直接开 `http://127.0.0.1:8000/` 会 404，那是路径问题，不是程序坏了。

三、手动启动前后端（想理解每一步时看）

第 0 步 · 装依赖（只需一次）

```
cd /d <解压目录>\campus-ai\backend
python -m pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

第 1 步 · 启动后端（必须，前端要连它）

```
cd /d <解压目录>\campus-ai\backend
python -m uvicorn main:app --reload --port 8000
```

看到 `Uvicorn running on http://127.0.0.1:8000` 即为成功。**这个窗口要一直开着。**

第 2 步 · 启动前端（二选一）

方式 1 · 单独起静态服务（改前端代码刷新即见）

```
cd /d <解压目录>\campus-ai\frontend
python serve_nocache.py 5500 --bind 127.0.0.1
```

> **【注意】`serve_nocache.py` 的端口是「位置参数」**，写 `--port 5500` 会报参数错误。
> 也可以用 `python -m http.server 5500` 代替，只是它不带 no-cache 头，改了页面可能要强制刷新。

方式 2 • 干脆不起前端（最省事）—— 后端已经把前端托管在 `/app/` 下：

```
http://127.0.0.1:8000/app/login.html
```

端口与配置

服务	默认端口	改端口要同步改哪里
FastAPI 后端	`8000`	`frontend/js/config.js` 的 `API_BASE`
前端静态服务	`5500`	无需改（前端会自动推导后端地址）

> 后端端口读取顺序为 `CAMPUS_API_PORT → PORT → API_PORT → 8000`，云端部署时由平台注入 `PORT`，**不要在启动命令里写 `--port $PORT`**。

四、API Key 说明

全部 Key 已写入 `backend/.env`，**开箱即用**，无需自己去申请：

变量	用途	当前状态
`GLM_API_KEY`	智谱 GLM 大模型（对话、学习计划、文档问答）	已配置
`GLM_MODEL` / `GLM_VISION_MODEL`	文本模型与视觉模型名	已配置
`GLM_BASE_URL`	智谱 OpenAI 兼容接口地址	已配置
`AMAP_WEB_KEY`	高德地图 Web 服务（路径规划、天气、地理编码）	已配置
`QDRANT_URL` / `QDRANT_API_KEY`	Qdrant 向量库（情绪陪伴的长期记忆 RAG）	已配置
`DATABASE_URL`	数据库地址	**留空** → 自动用本地 SQLite `backend/campus_ai.db`

换学校 / 换 Key 怎么做：复制 `backend/.env.example` 为 `.env`，把值换成自己的即可。`example` 里每个变量上方都有注释说明去哪儿申请。

五、文件结构 • 每个文件的作用

```
campus-ai/
├── start-windows.bat 等启动脚本
├── backend/                        FastAPI 后端
│   ├── main.py / config.py / database.py
│   ├── models/                     数据模型
│   ├── routers/                    接口路由（6 组）
│   └── services/                   外部服务与 AI 能力封装
├── frontend/                       零构建原生多页面前端
│   ├── *.html                      13 个页面（另有 nav.html 是顶栏片段，不单独打开）
│   ├── css/style.css
│   ├── js/                         按页拆分的原生 ES 模块
│   └── img/                        图片资源
├── tools/                          测试与质量工具链（可删，不影响运行）
└── 启动工具/                       同一套启动脚本的「就点这个文件夹」版
```

5.1 根目录

文件	作用
`start-windows.bat`	**一键启动器。**自动探测 Python → 补装依赖 → 起后端(8000) + 前端(5500) → 打开浏览器。端口被占用时自动在 8000-8020 / 5500-5520 里挑空闲端口
`check-windows.bat`	**环境自检。**检查 Python 版本、依赖是否装齐、端口占用情况；只输出信息，绝不改动任何文件
`check-lan.bat`	**局域网自检。**手机打不开页面时跑它，用于区分「网卡选错 / 防火墙拦截 / 不在同一网络」三种原因
`使用说明.pdf`	本文件
`.gitattributes`	Git 属性：规定换行符与二进制文件的处理方式，保证 `.bat` 跨系统拉取时不被改写
`.gitignore`	Git 忽略规则：排除数据库、缓存、密钥等不该进版本库的文件
`jsconfig.json`	VS Code 的 JS 类型检查配置（覆盖 `frontend/js/**/*.js`）

5.2 `backend/` —— FastAPI 后端

配置与入口

文件	作用
`.env`	**真实环境变量，Key 已填好** —— 「解压即用」靠它
`.env.example`	同上的模板，值留空，换 Key 时照着改
`build.txt`	**构建号唯一来源**（当前 `2026-11-01-a`）。改号用 `tools/bump_build.py`，它会一处改、全项目同步（约 120 处前端 `?v=` 引用）
`main.py`	**FastAPI 入口。**挂载 6 组路由、把前端静态页托管到 `/app`、启动时建表与写入种子数据
`config.py`	全局配置。读取 `.env`、解析监听端口（`CAMPUS_API_PORT → PORT → API_PORT → 8000`）、CORS 白名单
`database.py`	SQLAlchemy 引擎与会话管理。默认 SQLite，可平滑切 MySQL（只改 `DATABASE_URL`）
`errors.py`	全局异常处理。把 FastAPI / Pydantic 的英文报错统一翻译成中文提示
`requirements.txt`	Python 依赖清单。**刻意全写成 ASCII** —— 否则 pip 在中文 Windows 的 GBK 控制台会解码失败
`schema.sql`	建表 SQL（参考用；实际建表由 ORM 完成）
`seed.py`	种子数据：空白时间表 6:00–23:00（共 18 个节次，含南/北区作息差）

数据模型

文件	作用
`models/orm.py`	ORM 表模型：用户、时间槽、课程、情绪对话、测评结果（共 5 张表）
`models/schemas.py`	Pydantic 请求/响应模型，定义每个接口的入参与出参

接口路由

文件	作用
`routers/auth.py`	注册 / 登录 / 注销 / 账号设置。导出 `get_current_user` 鉴权依赖供其它路由复用
`routers/planner.py`	学业规划：时间槽查询、课程与活动增删改、Excel / CSV 导入、AI 生成学习计划
`routers/emotion.py`	情绪陪伴：AI 倾听对话（GLM + Qdrant 长期记忆）、情绪测评问卷与结果
`routers/map_weather.py`	校园地图点位、步行路线规划、天气查询

服务层（外部能力封装）

文件	作用
`services/study_agent.py`	**学业规划 AI Agent**：function calling 编排 6 个工具（读时间表 / 查节次作息 / 算空闲时段 / 归一楼名 / 查天气 / 检索学校文档）+ 系统提示词。**两个人格**：`plan` 调工具排时间表、`explain` 是学习教练（不调工具、不排表，只讲方法与思路）
`services/timetable_excel.py`	课表 Excel（`.xls` / `.xlsx`）解析与容错：4 种表头写法、4 种星期写法、2 种连堂排版、楼名口语归一
`services/vision.py`	课表截图识别的模型编排（代码保留，当前界面未开放入口）
`services/glm.py`	智谱 GLM 调用封装（OpenAI 兼容协议），统一超时与重试
`services/llm_json.py`	从大模型返回里稳健解析 JSON 的小工具（planner 与 vision 共用）
`services/qdrant.py`	Qdrant Cloud 封装：情绪陪伴的长期记忆（RAG），按 `user_id` 硬隔离
`services/amap.py`	高德 Web 服务封装：天气、地理编码、步行路径规划
`services/scan_login.py`	扫码登录链路（演示级）：手机扫码 → 网页自动登录
`services/data_export.py`	数据「输出」：在删除之前把库数据与向量库数据导出并打印

5.3 `frontend/` —— 零构建原生多页面前端

页面（11 个 `.html`）

文件	作用
`index.html`	首页：功能导览、轮播、顶部统计
`login.html`	登录 / 注册页，含「扫码登录」标签（二维码轮询）
`planner.html`	**时间表**（学业规划主入口）：看/改时间表、导入、按周浏览
`reminder.html`	**日程提醒**：今天明天的行程清单，可点课程直达导航
`plan.html`	**AI 伴学**：对话式把目标与时间表交给 Agent，排未来几天的作息
`emotion-chat.html`	情绪陪伴·AI 倾听对话（不含测评）
`emotion-quiz.html`	情绪测评问卷（8 题）
`emotion-result.html`	测评结果报告页
`account.html`	账号设置：改昵称 / 改密码 / 注销账号
`nav.html`	**校园导航页**：高德地图、点位类型筛选、起终点步行导航（走校内路网，不走公共机动车路网）
`scan-confirm.html`	手机端「确认登录」页。**旧版兜底路径**，现主流是扫码后直接在 `login.html` 登录

样式与静态服务

文件	作用
`css/style.css`	全站唯一样式表：布局、配色、响应式与移动端瘦身规则
`serve_nocache.py`	带 `no-cache` 的本地静态服务器，替代 `python -m http.server`（改了页面不用强制刷新）。**端口是位置参数**
`jsconfig.json`	前端目录的编辑器 JS 类型检查配置
`favicon.ico` / `favicon.png`	站点图标

脚本 `js/`

文件	作用
`common.js`	前端公共层：顶栏渲染（各页统一 4 个入口）、登录态「门口拦截」、接口封装（同时带 `X-Campus-Token` 与 `Authorization` 两个头）、Toast、HTML 转义、高德 SDK 加载
`config.js`	前端配置：`API_BASE` 自动推导（保证换电脑、换目录都不用改） + 高德 Key
`nav.js`	地图导航页逻辑：高德地图、校园点位、起终点步行导航、天气提示
`campus.js`	校园共享数据：作息时间表 + 教学楼（10 栋，区分南区/北区）
`campus-graph.js`	校内步行路网图数据与算法（节点 / 路段 / 最短路），导航「680 米」的来源
`icons.js`	内联 SVG 图标集 —— 不依赖 emoji 字体，任何系统都能正确显示
`login.js`	登录页逻辑：账号登录、注册、扫码登录（含二维码轮询与票据兑换）
`account.js`	账号设置页逻辑
`planner-core.js`	学业规划公共核心，三个子页共用：时间槽加载、数据读写、视图刷新
`planner-timetable.js`	时间表子页：表格渲染、导入（截图/Excel/JSON/CSV）、核对弹窗、增删改
`planner-reminder.js`	日程提醒子页：按天聚合、通知、导航入口
`planner-plan.js`	AI 伴学子页：对话交互、调用记录展示、计划渲染与保存
`emotion-chat.js`	情绪对话页逻辑（本地历史 + 后端记忆）
`emotion-quiz.js`	情绪测评作答与提交
`emotion-result.js`	测评结果渲染
`quiz-data.js`	测评题库与评分逻辑（题目、选项、分级、建议文案）
`qrcode.js`	纯前端二维码生成器（零依赖、可离线）
`scan-confirm.js`	手机确认登录页逻辑
`types.js`	全局 JSDoc 类型定义，**仅供编辑器提示，运行时无副作用**

图片 `img/`

文件	作用
`hero-gate.jpg` / `hero-library.jpg`	首页轮播主图
`feature-planner.png`	首页功能卡配图：学习规划
`feature-emotion.png`	首页功能卡配图：情绪陪伴
`feature-map.png`	首页功能卡配图：校园导航

5.4 `tools/` —— 测试与质量工具链

> 这一层**删掉也不影响网站运行**，它是工程质量的自证材料（300+ 项自动化断言的来源）。
> 需要 Node.js 才能跑 `.mjs`，纯 Python 的那些不需要。

构建与静态检查

文件	作用
`bump_build.py`	统一构建号：一处输入，全项目同步（含前端资源版本号）
`check_bat.py`	`.bat` 静态检查：找出可能导致「双击闪退」的常见语法问题
`check_encoding.py`	中文文本文件编码校验（统一 GBK）
`sync_launcher.py`	把 `start-windows.bat` 整份同步到 `启动工具/start.bat`，保证两者逐字节一致
`gen_structure.py`	重建「包内结构.txt」（本包已移除该清单，工具仍保留备用）
`verify.js`	改完代码后的自动校验（语法检查等）
`behavior_test.js`	两个「真跑一遍」的浏览器行为验证（首页轮播、导航），防止只看代码以为没问题

图片工具

文件	作用
`crop_png.py` / `shrink_png.py`	PNG 裁切 / 压缩
`make_hero.py`	生成首页 hero 图
`lookup_buildings.py`	按关键词查教学楼数据的小工具

路网质量守卫

文件	作用
`qa_campus_graph.py`	校内路网「贴路」硬指标守卫：离路 p90、越界里程占比（本包「3.27 米 / 5.7%」的来源）
`qa_campus_graph.png`	上面这个守卫跑出来的可视化结果
`refdata/campus_shot.json`	路网守卫的参考数据
`refdata/campus_shot.png`	路网守卫的参考底图

自动化断言（按被测对象分组）

文件	作用
`test_site_smoke.mjs`	全站回归：登录后逐页打开，断言无控制台错误、无白屏
`test_auth_token.mjs`	鉴权 token 传递的契约测试（两个请求头、401 语义区分）
`test_api_base.mjs`	前端 API 地址推导的回归测试（换电脑/换目录不改配置）
`test_lan_ip.py`	批处理里「挑哪张网卡」那段逻辑的离线单测
`test_launcher_url.py`	启动器 URL 规则的离线单测
`test_campus_graph.mjs`	校内步行路网纯逻辑守卫（不需要浏览器、不需要联网）
`test_campus_places.mjs`	校园点位数据契约测试
`test_map_labels.mjs`	地图标签显示规则 + 类型筛选的验收测试
`test_term_timetable.mjs`	「时间表」术语契约测试（正向 + 反向双查，防旧文案回潮）
`test_timetable_freeze.mjs`	时间表「表头 + 节次列冻结」验收测试
`test_xls_import.mjs`	课表 Excel（`.xls` / `.xlsx`）导入的端到端回归
`test_plan_chat.mjs`	AI 伴学对话页的契约测试
`test_mobile_login.mjs`	移动端登录最终验收
`test_scan_flow.mjs`	扫码登录端到端验证（双浏览器 context）
`test_scan_login_contract.py`	扫码登录契约测试
`test_study_agent.py`	学业规划 Agent 测试
`test_data_export.py`	数据导出功能测试

评测与无障碍（构建 `2026-11-01-a` 扩展）

文件	作用
`eval_agent.py`	Agent 评测。`python tools/eval_agent.py` 只跑路由（秒级、不花钱）；加 `--live` 真跑接口，出延迟 / token / 成本 / 工具选择等指标，报告写进 `reports/`
`test_rag_docs.py`	学生手册问答的守卫测试（83 项）：语料是否可追溯、切块是否丢字、假命中是否被拦、**检索为空时是否真的没调模型、问的动作在片段里没出现时会不会乱套别的流程**
`audit_a11y.mjs`	无障碍审计。用 axe-core 按 WCAG 2.1 A/AA 扫全站页面（浅色 + 黑夜两轮），serious/critical 违规会让脚本以非 0 退出

5.5 `启动工具/` —— 「就点这个文件夹」版

文件	作用
`start.bat`	一键启动。与根目录 `start-windows.bat` **逐字节一致**（由 `tools/sync_launcher.py` 强制同步）
`check.bat`	环境自检，对应根目录的 `check-windows.bat`
`check-lan.bat`	局域网自检，手机连不上时用

> 为什么脚本里全是英文？两个原因：① `.bat` 用中文会被 Windows 控制台代码页弄乱、甚至直接报错退出（「双击一闪就没」）；② `pip` 读 `requirements.txt` 时按系统默认编码解码，中文注释会触发 `UnicodeDecodeError`。

六、常见问题速查

现象	原因与处理
双击 `.bat` 一闪就没	Python 没装或不在 PATH。双击 `check-windows.bat` 看具体缺什么
首次启动卡在装依赖	正常，约 1–2 分钟。装完会自动继续
页面某处显示「网络错误」	后端窗口是不是被关了？确认 `CampusAI-Backend` 那个窗口还开着
打开 `http://127.0.0.1:8000/` 报 404	少了 `/app/` 前缀，应为 `/app/login.html`
手机打不开	① 手机和电脑要在**同一个 Wi-Fi**；② 跑 `check-lan.bat` 看是否被防火墙拦；③ 用脚本打印的**单端口**地址（`http://电脑IP:8000/app/login.html`）
改了前端代码页面没变	用 `serve_nocache.py` 起前端；或按 Ctrl+F5 强制刷新
换了新压缩包，数据还在吗	在。只在 `backend/campus_ai.db.bak-*` 里留一份备份，不会清库
想让数据从零开始	删掉 `backend/campus_ai.db` 再启动，或启动前设 `CAMPUS_RESET_DB=1`

七、作品概要

> **核心能力：查、算、排、记、走。**

时间表导入 → 日程提醒 → AI 排学习计划 → 情绪陪伴 → 校园步道导航，四个模块共用同一份数据、构成一个闭环。 其中校园导航不依赖公共地图的机动车路网，而是**自建了校内步行路网**：从知行楼到平远楼， 公共地图给 1675 米，本作品给 **680 米。**

八、AI 伴学的「学校规定问答」怎么用

该模块已扩充至 29 篇语料：**回答前先检索学生手册，并在答案中标注出处。**

8.1 能问什么

在「AI 伴学」对话框中直接输入问题即可：

示例问法	系统回应
休学需要准备什么材料?	列出材料清单、走哪个系统、找哪个办公室，末尾标 `来源：《休学手续办理流程》`
转专业要什么条件?	同上，走「一站式服务平台」的路径一步不落
图书馆几点开门? / 专座怎么预约?	按手册原文给时间与步骤，末尾标 `来源：《图书馆开放时间》`
宿舍坏了怎么报修? / 校园网密码忘了?	给出「完美校园」里的具体入口路径、或密码找回的 6 个步骤
什么时候放寒假? / 第 10 周是哪几天?	直接给日期（校历里查得到）
辅修可以大二报名吗?	按手册原文回答

另外三类问题**根本不用等模型**（下文 8.5 会解释为什么）：问天气、问某栋楼在哪、 问某个时间有几栋教学楼 —— 都是 1 秒内出答案。

8.2 不做的部分（边界比能力更重要）

手册未收录的内容不会被编造。

询问「校园卡丢了怎么补」，系统会明确回答「学校手册中未收录校园卡丢失的补办流程」， 并建议联系教务处（0591-83969194）或辅导员，而不会参照「毕业证明补办」流程 生成一份形式上相似的答案。这是本作品的取舍：**宁可不答，也不输出会导致无效往返的流程。**

技术上保证了这一点，而且是**两道**保障：

1. 检索不到相关内容时，程序**根本不会调用大模型**，直接给出上面那句固定说法。
2. 检索到了、但只是「同主题的另一件事」时（问「补办」却检索到《一卡通线上充值指南》，

两件事都含「校园卡」三个字），程序先从问句中提取动作词 （补办 / 挂失 / 报修 / 预约 / 打印…），再到检索片段里逐个核对， 若均未出现，则将该结论写入提示词：**这段片段里没有用户问的这个操作**， 并禁止它标注出处、禁止它自己提一个办事窗口的名字。

换句话说：**判断不交给模型，模型只负责把已经确认过的结论说人话。**

8.3 语料从哪来、怎么更新

- 来源：三个部门官网的公开页面 + 两个官方公众号推文，共 **29 篇**（23 168 字），

全部放在 `backend/data/campus_docs/`，每篇开头都带原文链接与抓取日期，可逐条核对：

- 阳光学院教务处（`jwc.ygu.edu.cn`）：**学生办事指南 13 项 + 2026-2027 学年校历**
- 阳光学院图书馆：**11 篇**（开放时间、读者须知、借阅流程、馆藏分布、常见问题、

空间使用、专座、场地借用、自助借还、数字资源、图书馆管理办法）

- 阳光学院后勤 / 信息化（`sbc.ygu.edu.cn` 及官方公众号）：**4 篇**（后勤工作职责、

校园报修系统使用指南、一卡通线上充值指南、校园网准入系统密码找回）

- 抓不到的一律不收：整页是列表的、整页是图片的、整页只有一个下载按钮的，

全都剔除 —— 这些页面切出来的片段答不了任何问题，留在库里只会稀释检索结果。

- 重新导入（改完语料后执行，可重复跑，不会重复堆叠）：

```
cd backend
python -m services.rag_docs --dry-run   # 先看会切多少块，不写库
python -m services.rag_docs             # 真正写入向量库
```

8.4 想知道它测得怎么样

见包内 `docs/agent-eval.md`：**78 + 21 条**评测集（21 条留出集未参与调参）、 离线路由与在线端到端两层跑法、失败样例与成本核算都写在那里，`reports/` 下是原始 JSON 报告。

本版实测（构建 `2026-11-01-a`）：

口径	数字
路由准确率（离线，78 条）	**78/78 = 100%**（本版新增 10 条口语用例）
留出集准确率（21 条，未调参）	**21/21 = 100%**
在线端到端成功率	**15/15**
在线延迟（混合口径）	P50 22.7 秒 / P95 49.2 秒
真要排表的 8 条	P50 24.8 秒，token 中位 9.3 k
查手册 / 查天气这类	查手册 4.7–8.9 秒；查天气、问楼在哪 **0.5–0.8 秒、0 token**

> 「混合口径」是指把不花钱的快速路径也一起统计了 —— 它们的 token 恒为 0，
> 混在中位数里会把整体拉好看。所以上表把「真要排表」单独拆出来报，避免数字虚高。

8.5 为什么问天气、问楼在哪这么快

伴学在调模型之前先做一次**路由判定**，一共六条路：

路	什么时候走	做什么	实测
`quick`	问天气 / 问楼在哪 / 问校园有多少栋教学楼 / 问某节课几点上	**只跑一个工具（或零工具），套模板回答，完全不调模型**	0.5–0.8 秒，0 token
`schedule_add`	「周三下午两点加一节高数」这种明确往时间表加一条的	只写一条记录	秒级
`docs`	问学校规定、办事流程、校历	检索手册 + 模型作答，标出处	5–9 秒
`explain`	「怎么学 / 记不住 / 什么是…」（学习教练人格）	不调工具、不排表	十几秒
`local_ask`	「周四有没有课」「哪天有空」	本地直接查课表，**不调模型**	毫秒级
`plan`	真要排一份计划	调工具 + 多轮模型	十几到几十秒

这一版之所以要把 `quick` 单独拆出来，是因为用户实测反馈：**问天气、问楼在哪， 它会去调排表工具、还生成一份学习计划**（6 条标注里错 3 条）。 根因是原来只分五类，凡是「不走课表、不走学校规定、不走学习教练」的句子全落到 `plan`， 而 `plan` 的主流程就是排表。

拆出来之后不只是变快，更重要的是**不让模型有机会编**：楼在南北区是查楼名表得来的事实， 不是语言模型的推理题；查不到就直说查不到，不给「大概在附近」这种猜测。

8.6 源码仓库与持续集成（可点开核对）

源码在 GitHub 上公开：**https://github.com/1280507078/campus-ai**

仓库首页顶部有一枚 CI 徽章，点开就是每次提交的构建记录。三个作业全部通过：

作业	做什么	本次结果
后端契约测试 + 覆盖率	手册问答 83 项、伴学 82 项、鉴权 27 项、数据导出 22 项	全过，覆盖率报告作为构建产物可下载
前端纯逻辑守卫	全部脚本语法检查 + 5 个静态守卫（250 项断言）	全过
e2e + 无障碍审计	真浏览器逐页打开 + axe-core 扫 WCAG 2.1 A/AA	11 页逐页打开**控制台错误 0**；浅色 13 页 + 黑夜 3 页 **serious/critical 为 0**

> 上述三个作业运行于 GitHub 的 Ubuntu 环境，非本机结果。即：「在干净环境中可安装、可运行、页面无报错、无障碍不退化」均由**第三方环境验证**，而非自述。

另外两项可以自己复核的性能/无障碍证据：

口径	登录页	主页
性能	**100**	97
无障碍	**100**	100
最佳实践	**100**	100
SEO	**100**	100
LCP / CLS / TBT	0.6 s / 0.004 / 0 ms	1.2 s / 0 / 0 ms

Lighthouse 桌面端 preset，报告原文件在 `reports/lighthouse-login.report.html` 与 `reports/lighthouse-index.report.html`，双击就能看逐条明细。

镜像是**在 CI 里真构建**的：`docker build` 之后起容器、探 `/api/ping` 通了才算过， 不推任何 registry（也就不需要账号和密钥）。镜像里不含 `.env`，所以这一步顺带验证了 「没有 Key 也能起来、只是 AI 功能降级」这个设计。

CI 跑通的过程中修掉两处只在 Linux 上才炸的写法，一并记在这里，因为其表现类似「代码缺陷」，实际并非如此： ① 测试里用「Z 盘不存在」模拟写盘失败，Windows 上确实失败，但 Linux 里 `Z:` 只是一个普通目录名， 反而真的把文件写进去了 —— 后来改成拿一个普通文件当目录用，两个平台都必然失败； ② 几个脚本把浏览器路径写死成 `/root/.cache/...chromium-1208`，而 GitHub 机器的家目录是 `/home/runner`，浏览器目录名还带版本号 —— 改成不传路径、让 Playwright 自己找。